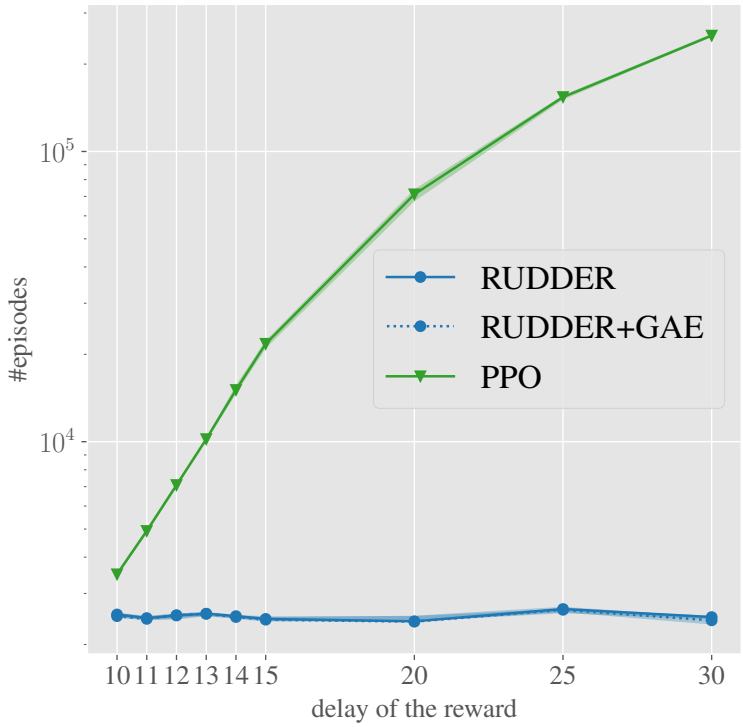

Figure A6: Comparison of performance of RUDDER with GAE (RUDDER+GAE) and without GAE (RUDDER) and PPO with GAE (PPO) on artificial task V with respect to the learning time in episodes (median of 100 trials) in log scale vs. the delay of the reward. The shadow bands indicate the 40% and 60% quantiles. Again, RUDDER significantly outperforms all other methods.

A4.2.1 Architecture

We use a modified PPO architecture and a separate reward redistribution model. While parts of the two could be combined, this separation allows for better comparison between the PPO baseline with and without RUDDER.

PPO architecture. The design of the policy and the value network relies on the *ppo2* implementation [21], which is depicted in Figure A7 and summarized in Table A4. The network input, 4 stacked Atari game frames [82], is processed by 3 convolution layers with ReLU activation functions, followed by a fully connected layer with ReLU activation functions. For PPO with RUDDER, 2 output units, for the original and redistributed reward value function, and another set of output units for the policy prediction are applied. For the PPO baseline without RUDDER, the output unit for the redistributed reward value function is omitted.

Reward redistribution model. Core of the reward redistribution model is an LSTM layer containing 64 memory cells with sigmoid gate activations, tanh input nonlinearities, and identity output activation functions, as illustrated in Figure A7 and summarized in Table A4. This LSTM implementation omits output gate and forget gate to simplify the network dynamics. Identity output activation functions were chosen to support the development of linear counting dynamics within the LSTM layer, as is required to count the reward pieces during an episode chunk. Furthermore, the input gate is only connected recurrently to other LSTM blocks and the cell input is only connected to forward connections from the lower layer. For the vision system the same architecture was used as with the PPO network, with the first convolution layer being doubled to process Δ frames and full frames separately in the first layer. Additionally, the memory cell layer receives the vision feature activations of the PPO network, the current action, and the approximate in-game time as inputs. No gradients from the reward redistribution network are propagated over the connections to the PPO network. After the LSTM layer, the reward redistribution model has one output node for the prediction $\widehat{\mathbf{g}}$ of the return realization $\mathbf{g}$ of the return variable G_0. The reward redistribution model has 4 additional output nodes for the auxiliary tasks as described in Section A4.2.3.

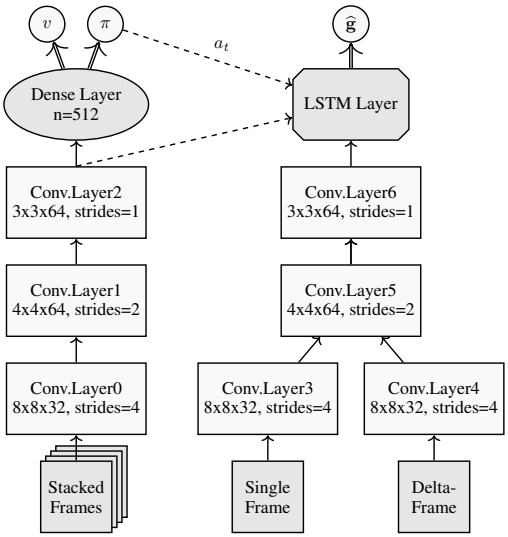

Figure A7: RUDDER architecture for Atari games as described in Section A4.2.1. Left: The *ppo2* implementation [21]. Right: LSTM reward redistribution architecture. The reward redistribution network has access to the PPO vision features (dashed lines) but no gradient is propagated between the networks. The LSTM layer receives the current action and an approximate in-game-time as additional input. The PPO outputs v for value function prediction and π for policy prediction each represent multiple output nodes: the original and redistributed reward value function prediction for v and the outputs for all of the available actions for π. Likewise, the reward redistribution network output $\widehat{\mathbf{g}}$ represents multiple outputs, as described in Section A4.2.3 Details on layer configuration are given in Table A4.

Layer	Specifications		Layer	Specifications	
Conv.Layer 0	features	32	Conv.Layer 4	features	32
	kernelsize	8x8		kernelsize	8x8
	striding	4x4		striding	4x4
	act	ReLU		act	ReLU
	initialization	orthogonal, `gain`=$\sqrt{2}$		initialization	orthogonal, `gain`=0.1
Conv.Layer 1	features	64	Conv.Layer 5	features	64
	kernelsize	4x4		kernelsize	4x4
	striding	2x2		striding	2x2
	act	ReLU		act	ReLU
	initialization	orthogonal, `gain`=$\sqrt{2}$		initialization	orthogonal, `gain`=0.1
Conv.Layer 2	features	64	Conv.Layer 6	features	64
	kernelsize	3x3		kernelsize	3x3
	striding	1x1		striding	1x1
	act	ReLU		act	ReLU
	initialization	orthogonal, `gain`=$\sqrt{2}$		initialization	orthogonal, `gain`=0.1
Dense Layer	features	512	LSTM Layer	cells	64
	act	ReLU		gate act.	sigmoid
	initialization	orthogonal, `gain`=$\sqrt{2}$		ci act.	tanh
Conv.Layer 3	features	32		output act.	linear
	kernelsize	8x8		bias ig	trunc.norm., `mean`= -5
	striding	4x4		bias ci	trunc.norm., `mean`= 0
	act	ReLU		fwd.w. ci	trunc.norm., `scale`= 0.0001
	initialization	orthogonal, `gain`=0.1		fwd.w. ig	omitted
				rec.w. ci	omitted
				rec.w. ig	trunc.norm., `scale`= 0.001
				og	omitted
				fg	omitted

Table A4: Specifications of PPO and RUDDER architectures as shown in Figure A7. Truncated normal initialization has the default values `mean`= 0, `stddev`= 1 and is optionally multiplied by a factor `scale`.

A4.2.2 Lessons Replay Buffer

The lessons replay buffer is realized as a priority-based buffer containing up to 128 samples. New samples are added to the buffer if (i) the buffer is not filled or if (ii) the new sample is considered more important than the least important sample in the buffer, in which case the new sample replaces the least important sample.

Importance of samples for the buffer is determined based on a combined ranking of (i) the reward redistribution model error and (ii) the difference of the sample return to the mean return of all samples in the lessons buffer. Each of these two rankings contributes equally to the final ranking of the sample. Samples with higher loss and greater difference to the mean return achieve a higher ranking.

Sampling from the lessons buffer is performed as a sampling from a softmax function on the sample-losses in the buffer. Each sample is a sequence of 512 consecutive transitions, as described in the last paragraph of Section A4.2.3.

A4.2.3 Game Processing, Update Design, and Target Design

Reward redistribution is performed in an online fashion as new transitions are sampled from the environment. This allows to keep the original update schema of the PPO baseline, while still using the redistributed reward for the PPO updates. Training of the reward redistribution model is done separately on the lessons buffer samples from Section A4.2.2. These processes are described in more detail in the following paragraphs.

Reward Scaling. As described in the main paper, rewards for the PPO baseline and RUDDER are scaled based on the maximum return per sample encountered during training so far. With i samples sampled from the environment and a maximum return of $\mathbf{g}_i^{\max} = \max_{1 \leqslant j \leqslant i}\{|\mathbf{g}_j|\}$ encountered, the scaled reward r_{new} is

$$r_{\text{new}} \;=\; \frac{10\, r}{\mathbf{g}_i^{\max}} \,. \tag{A266}$$

Goal of this scaling is to normalize the reward r to range $[-10, 10]$ with a linear scaling, suitable for training the PPO and reward redistribution model. Since the scaling is linear, the original proportions between rewards are kept. Downside to this approach is that if a new maximum return is encountered, the scaling factor is updated, and the models have to readjust.

Reward redistribution. Reward redistribution is performed using differences of return predictions of the LSTM network. That is, the differences of the reward redistribution model prediction $\widehat{\mathbf{g}}$ at time step t and $t-1$ serve as contribution analysis and thereby give the redistributed reward $r_t = \widehat{\mathbf{g}}_t - \widehat{\mathbf{g}}_{t-1}$. This allows for online reward redistribution on the sampled transitions before they are used to train the PPO network, without waiting for the game sequences to be completed.

To assess the current quality of the reward redistribution model, a quality measure based on the relative absolute error of the prediction $\widehat{\mathbf{g}}_T$ at the last time step T is introduced:

$$\texttt{quality} \;=\; 1 \;-\; \frac{|\mathbf{g} - \widehat{\mathbf{g}}_T|}{\mu} \, \frac{1}{1-\epsilon}, \tag{A267}$$

with ϵ as quality threshold of $\epsilon = 80\%$ and the maximum possible error μ as $\mu = 10$ due to the reward scaling applied. `quality` is furthermore clipped to be within range $[0, 1]$.

PPO model. The *ppo2* implementation [21] samples from the environment using multiple agents in parallel. These agents play individual environments but share all weights, i.e. they are distinguished by random effects in the environment or by exploration. The value function and policy network is trained online on a batch of transitions sampled from the environment. Originally, the policy/value function network updates are adjusted using a policy loss, a value function loss, and an entropy term, each with dedicated scaling factors [115]. To decrease the number of hyperparameters, the entropy term scaling factor is adjusted automatically using Proportional Control to keep the policy entropy in a predefined range.

We use two value function output units to predict the value functions of the original and the redistributed reward. For the PPO baseline without RUDDER, the output unit for the redistributed reward is omitted. Analogous to the *ppo2* implementation, these two value function predictions serve to compute the advantages used to scale the policy gradient updates. For this, the advantages for original reward a_o and redistributed reward a_r are combined as a weighted sum $a = a_o\,(1 - \texttt{qualityv}) + a_r\,\texttt{quality}$. The PPO value function loss term L_v is replaced by the sum of the value function v_o loss L_o for the original reward and the scaled value function v_r loss